# Gamma

IMPLEMENTATION GUIDE

ANTONIO FREIXAS • DECEMBER 31, 2021

## Execution Overview

A *dynamic* variable is a variable whose value can change during the execution of a script. Animation variables are dynamic variables; other types of dynamic variables may be added later. The execution process varies slightly depending on whether a script contains any dynamic variables.

The execution of a script proceeds as follows:

- Comments are stripped from the script.
- The script characters are grouped into tokens by the tokenizer.
- The tokens are parsed by the parser to produce an *h-code* program, a machine code program for a pseudo machine.
- If dynamic variables are present:
  - The h-code program is optimized. This process is described in detail later.
  - The execution process for optimized code is a bit different from non-optimized code; again, this is described later.
- The **display** command is executed. Most of the commands properties are applied only once; the exception is the **frame** property, which is applied to each execution of the h-code. This other properties set the diagram window's size, defines how the diagram reacts to window size changes by the user, and sets the initial zoom and pan.
- The h-code program is executed. The result is l-code, another machine code program for another pseudo machine.
- Coordinates in the l-code are changed from being relative to the default frame to being relative to the drawing frame.
- The l-code program is executed. This actually draws the diagram.

The l-code program is re-executed whenever the user zooms or pans.

The h-code program is re-executed whenever a dynamic variable changes. For animations, this occurs when the next animation frame is drawn.

## Execution Details

### Pre-processing

Comments are replaced by the empty string.

### Tokenizer

The tokenizer groups characters into tokens, which have a type and a value.

| Type | Value |
|---|---|
| NUMBER | A floating point value (not including the sign). |
| STRING | The character string represented by the characters in the script (outer quotes removed, quoted characters processed). |
| OPERATOR | The operator character: **+** \| **-** \| ***** \| **/** \| **^** |
| NAME | The name. |

### Gamma

| Type | Value |
|---|---|
| DELIMITER | The delimiter character: **;** \| **,** \| **:** \| **.** \| **=** \| **(** \| **)** \| **[** \| **]** \| **<** \| **>** |

### Parser

The parser processes the tokens to produce the h-code progam.

#### *H-code Program*

The program consists of a linear sequence of values and instructions called h-code.

The sequence is viewed as something of a stack.  To execute the program, we begin with the first item on the list (this is the top of the stack), skipping over values until we reach an h-code instruction. This instruction operates on a number of the values above it, the number being determined by the specific h-code. The result of the operation is to perform an action or to produce a new value. In either case, the h-code instruction and the values it operated on are removed from the list. If the h-code returns a value, it is placed on the list at the position formerly occupied by the h-code.

An h-code instruction should never consume another h-code item. If it attempts to, an error occurs. When the last h-code instruction completes, the program should be totally consumed. If the list is not empty, an error occurs.

Following our semantic rules, only the last **display** and last **animation** commands have any effect. If, during parsing, either of these commands is encountered, the parser will check for a prior instance. If found, it is removed from the program. Whether found or not, the current command is added to the program and a pointer is retained in case that command appears later in the script.

#### *Values*

When creating an h-code program, all values look the same: they point to a structure that includes the value's type and the value.

Variables used as values require an extra step: dereferencing. This is done with the FETCH h-code whose output is the referenced value.

The ASSIGN h-code is the opposite of FETCH: it takes a value and stores it in a variable.

Object properties (dereferenced using a ".") need special handling. When used as a value, they are dereferenced using the DEREF h-code, which requires an object and a name (dereferencing works on objects, so when dereferencing a variable, it must first be FETCHed).

When an object property is assigned to a special assignment is required. This takes two strings: the first is the name of a variable (which must exist) and the second is the name of an object property (which must be an externally accessible property of the given variable). Unlike variables, the property must exist and its type cannot be changed. If the value is of the wrong type, an error occurs.

#### *Dynamic Value Assignments*

#### *H-Codes*

The following is the list of all h-codes. Values must be of the type listed or an error occurs.

LINE_INFO

This h-code has no arguments and produces no results. It tells the h-code engine which source script file and line is associated with the following h-codes. This information is used for error messages.

| LINE_INFO |
|---|

### SET_PRECISION

This h-code has no arguments and produces no results. It sets the precision of printed floating point numbers to either the display precision or the print precision specified by the **set** statement.

| |
|---|
| SET_PRECISION |

### FETCH

Fetch the value of a variable.

- 1 is the name of the variable to access.

An error occurs if the name is not found in the symbol table. The result is the value stored in the variable.

| | |
|---|---|
| **1** | String |
| | FETCH |

### ENABLE

Enable or disable execution of selected statements and commands.

- 1 is the value used to determine whether to enable or disable statements and commands. It is treated as a boolean.

This h-code produces no results.

| | |
|---|---|
| **1** | Float |
| | PRINT |

### PRINT

Print a value to a dialog.

- 1 is the value to print.

Every primitive and object type accessible to the user can be printed.

This h-code produces no results.

| | |
|---|---|
| **1** | Object |
| | PRINT |

### FETCH_PROP

Fetch the value of an externally-accessible object property.

- 1 is the value whose property's value we want.
- 2 is the name of the property to access.

Look at the value to see if it has an externally accessible property with the given name. If no property with that name exists, an error occurs.

The result is the value of the property.

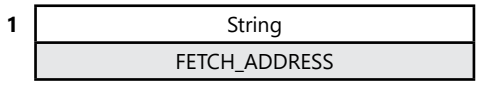
| | |
|---|---|
| **1** | Value |
| **2** | String |
| | FETCH_PROP |

### FETCH_ADDRESS

Fetch the value of a variable.

- 1 is the name of the variable to access.

If the variable does not exist, create it. The result is the address, which is a pointer to variable.

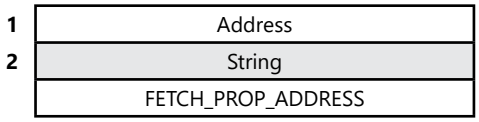



## FETCH_PROP_ADDRESS

Fetch the address of an externally-accessible object property.

- 1 is an address, a pointer to a variable in the symbol table or in some other value.
- 2 is the name of the property to access.

The result is the address of the property, which is a pointer to the property variable. If the property does not exist, an error occurs.



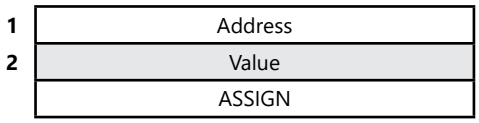

## ASSIGN

Assign a value.

- 1 is the address where the value should be stored.
- 2 is the value to assign.

If the address points to top-level variable, its current value and type are replaced by the new value and type. If it points to an object property, an error occurs if the property's type is not the same as the value's type.

This h-code produces no result.



## ANIM_ASSIGN

Create or update an animation variable and assign it its initial value.

- 1 is the address of the where the initial value of the animation variable should be stored. The address cannot point to an object property.
- 2 is the initial value of the animation variable. It must be a float.
- 3 is the limit value. If no limit was given, it is NaN.
- 4 is the step value.

During the first execution of the h-code program, this h-code sets up the animation variable. On subsequent frames, it does nothing (other than remove itself from the stack). The animation system will pre-load the variable and the appropriate value into the symbol table.

This h-code produces no result.

| | |
|---|---|
| **1** | Address |
| **2** | Float |
| **3** | Float |
| **4** | Float |
| | ANIM_ASSIGN |

## RANGE_DISPLAY_ASSIGN

Create or update a range display variable and assign it its initial value.

- 1 is the address of the where the initial value of the range display variable should be stored. The address cannot point to an object property.
- 2 is the initial value of the range display variable. It must be a float.
- 3 is the minimum value of the range display variable. It must be a float.
- 4 is the maximum value of the range display variable. It must be a float.
- 5 is the range display's label. It can be any displayable value, but will be converted to a string.

During the first execution of the h-code program, this h-code sets up the dynamic variable. On subsequent frames, it does nothing (other than remove itself from the stack). The display system will pre-load the variable and the appropriate value into the symbol table.

This h-code produces no result.

| | |
|---|---|
| **1** | Address |
| **2** | Float |
| **3** | Float |
| **4** | Float |
| **5** | Object |
| | RANGE_DISPLAY_ASSIGN |

## BOOLEAN_DISPLAY_ASSIGN

Create or update a boolean display variable and assign it its initial value.

- 1 is the address of the where the initial value of the boolean display variable should be stored. The address cannot point to an object property.
- 2 is the initial value of the range display variable. It must be a float.
- 3 is the boolean display's label. It can be any displayable value, but will be converted to a string.

During the first execution of the h-code program, this h-code sets up the dynamic variable. On subsequent frames, it does nothing (other than remove itself from the stack). The display system will pre-load the variable and the appropriate value into the symbol table.

This h-code produces no result.

| | |
|---|---|
| **1** | Address |
| **2** | Float |
| **3** | Object |
| | RANGE_DISPLAY_ASSIGN |

## UNARY_PLUS

This does nothing.

- 1 is a value.

The result is the value given.

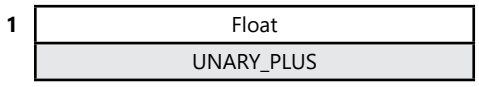
| | |
|---|---|
| **1** | Float |
| | UNARY_PLUS |

### UNARY_MINUS

Negate the given value.

- 1 is a value.

The result is the negation of the value given.



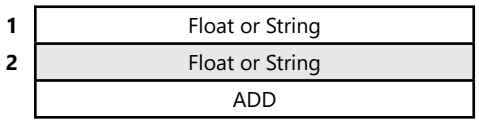

### ADD

Combine two values.

- 1 is the first value.
- 2 is the second value.

If one value is a float and the other a string, the float is converted to a string.

If the two values are floats, they are added. The result is the sum. If the two values are strings, the result is a new string, the concatenation of 2 onto 1.

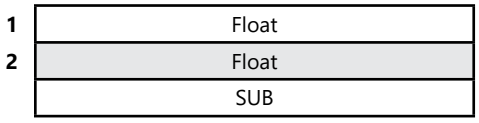



### SUB

Subtract.

- 1 is the first value.
- 2 is the second value.

Subtract 2 from 1. The difference is returned.



**Gamma**

### MULT

Multiply.

- 1 is the first value.
- 2 is the second value.

The product is returned.

| | |
|---|---|
| **1** | Float |
| **2** | Float |
| | MULT |

### DIV

Divide.

- 1 is the first value.
- 2 is the second value.

Divide 1 by 2. The quotient is returned. A divide by 0 generates an error.

| | |
|---|---|
| **1** | Float |
| **2** | Float |
| | DIV |

### EXP

Exponentiate.

- 1 is the first value.
- 2 is the second value.

Raise value 1 to the power of value 2. The result is returned.

| | |
|---|---|
| **1** | Float |
| **2** | Float |
| | EXP |

### LORENTZ

Perform a Lorentz transformation.

- 1 is a coordinate.
- 2 is a frame object.

Transform the coordinate from the default frame to the given frame. The resulting coordinate is returned.

| | |
|---|---|
| **1** | Coordinate |
| **2** | Frame |
| | LORENTZ |

### I

## NV_LORENTZ

Perform a inverse Lorentz transformation.

- 1 is a coordinate.
- 2 is a frame object.

Transform the coordinate from the given frame to the default frame. The resulting coordinate is returned.

| | |
|---|---|
| **1** | Coordinate |
| **2** | Frame |
| | EXP |

## FUNCTION

Call a function. String n is the name of an existing function.

- 1 - n -2 are the function arguments in reverse order.
- n - 1 is the function name.
- n is the number of arguments (including the name)

If the function does not exist, an error occurs. If the number of arguments supplied does not match the number of arguments required by the function, an error occurs. If the type of any argument does not match the type required, an error occurs.

The result is the function's output value. The value's type depends on the function.

| | |
|---|---|
| **1** | Arg 1 |
| **2** | Arg 2 |
| | ... |
| **n-3** | Arg m - 1 |
| **n-2** | Arg m |
| **n-1** | String |
| **n** | Integer (m+1) |
| | FUNCTION |

## W_INITIALIZER

Create a wordline initializer. The parser will need to do a bit of pre-processing to produce this h-code instruction. Worldline initializers can be **origin**, **distance**, and **tau**. The syntax allows these to be given multiple times and in any order.

If the same item is repeated, an error occurs—it is syntactically, but not semantically, allowed.

Once a wordline segment is encountered (or the worldline definition ends), any missing initializer is given its default value.

- 1 is a coordinate representing the origin. If no origin was given, use coordinate (0, 0).
- 2 is the tau value. If the tau value is not given, use 0.
- 3 is the distance value. If the distance value is not given, use 0.

The result is a worldline initializer object.

| | |
|---|---|
| **1** | Coordinate |
| **2** | Float |
| **3** | Float |
| | W_INITIALIZER |

**Gamma**

## W_SEGMENT

Create a a single wordline segment.

- 1 is the velocity value. If no velocity was given, use NaN.
- 2 is the acceleration value. If no acceleration was given, use 0.
- 3 is the limit type: T, TAU, D, or V. If no limit was given, use NONE.
- 4 is the limit value. If no limit was given, use NaN.

The result is a worldline segment object.

| | |
|---|---|
| **1** | Float |
| **2** | Float |
| **3** | Type |
| **4** | Float |
| | W_SEGMENT |

## OBSERVER

Create a new observer object.

- 1 is the worldline initializer.
- 2 - n-1 are worldline segments, with 2 being the first segment and n-1 being the last.
- n is the number of arguments, where m is the number of segments. m can be 0.

The result is an observer object.

| | |
|---|---|
| **1** | Worldline Initializer |
| **2** | Worldline segment 1 |
| **3** | Worldline Segment 2 |
| | ... |
| **n-2** | Worldline Segment m-1 |
| **n-1** | Wordline Segment m |
| **n** | Integer (m + 1) |
| | OBSERVER |

## FRAME

Create a new frame object.

- 1 an origin
- 2 is a velocity
- 3 is the **at** value. if no at clause was given, use value 0.0.

The result is a frame object.

| | |
|---|---|
| **1** | Coordinate |
| **2** | Float |
| | FRAME |

### OBSERVER_FRAME

Create a new frame object.

- 1 is an observer object.
- 2 is the **at** type: TIME, TAU, DISTANCE, or V. If no at clause was given, use type TAU.
- 3 is the **at** value. if no at clause was given, use value 0.0.

The result is a frame object.

| | |
|---|---|
| **1** | Observer |
| **2** | Type |
| **3** | Float |
| | FRAME |

### AXIS_LINE

Create a new line. The line is specified as parallel to an axis and crossing an offset on the opposite axis.

- 1 is the axis type: X or T.
- 2 is a frame object.
- 3 is a float representing the axis offset. If none was give, 0 is used.

The result is a line object.

| | |
|---|---|
| **1** | Type |
| **2** | Frame |
| **3** | Float |
| | AXIS_LINE |

### ANGLE_LINE

Create a new line. The line is specified by an angle and a point.

- 1 is the angle of the line.
- 2 is a coordinate that specifies the point through which the line goes.

The result is a line object.

| | |
|---|---|
| **1** | Float |
| **2** | Coordinate |
| | ANGLE_LINE |

### ENDPOINT_LINE

Create a new line. The line is specified by two endpoints.

- 1 is the first coordinate.
- 2 is the second coordinate.

The result is a line object.

| | |
|---|---|
| **1** | Coordinate |
| **2** | Coordinate |
| | ENDPOINT_LINE |

**Gamma**

## PATH

Create a new path.

- 1 - n-1 are the path coordinates.
- n is the number of arguments, where m is the number of segments.

| | |
|---|---|
| **1** | Coordinate 1 |
| **2** | Coordinate 2 |
| | ... |
| **n-2** | Coordinate m-1 |
| **n-1** | Coordinate m |
| **n** | Integer (m + 1) |
| | Path |

## BOUNDS

Create a bounding box.

- 1 is the first coordinate.
- 2 is the second coordinate.

The result is a bounding box.

| | |
|---|---|
| **1** | Coordinate |
| **2** | Coordinate |
| | BOUNDS |

## INTERVAL

Create an interval.

- 1 is the interval type, T, TAU or D.
- 2 is one end of the range.
- 3 is the other end of the range.

| | |
|---|---|
| **1** | Type |
| **2** | Float |
| **3** | Float |
| | INTERVAL |

## COORDINATE

Create a new coordinate.

- 1 is the x coordinate
- 2 is the t coordinate.

The result is a coordinate object.

| | |
|---|---|
| **1** | Float |
| **2** | Float |
| | COORDINATE |

## PROPERTY

Create a property structure.

- 1 is the name of the property.
- 2 is the property's value.

The validity of the name and value is not checked at this point.

| | |
|---|---|
| **1** | String |
| **2** | Object |
| | PROPERTY |

### PROPERTY_LIST

Create a property list structure. Again, the names and types are not checked at this point.

- 1 - n are either property objects or style objects.
- n is the number of arguments, where m is the number of values.

The result is a property list object.

| | |
|---|---|
| **1** | Value 1 |
| **2** | Value 2 |
| | ... |
| **n-2** | Value m -1 |
| **n-1** | Value m |
| **n** | Integer (m + 1) |
| | PROPERTY_LIST |

### STYLE

Create a new style object.

- 1 is the property list.

The result is a style object. Some error checking can occur at this point. Only style properties are allowed, and the value of the a property must match the required value.

| | |
|---|---|
| **1** | Property List |
| | STYLE |

### SET_STYLE

Set the default styles.

- 1 is the property list.

Some error checking can occur at this point. Only style properties are allowed, and the value of the a property must match the required value.

| | |
|---|---|
| **1** | Property List |
| | SET_STYLE |

This h-code produces no result. Instead, it creates a structure containing all style properties set to their factory default values. It then overwrites a property's value with the value of any matching property found in the given property list.

This structure is applied to the property lists received by the COMMAND h-code to produce the final property list (which is used when creating an l-code).

When an h-code program begins execution, a SET_STYLE command with an empty property list is implied.

## COMMAND

Execute a command.

- 1 is a property list (which could be empty).
- 2 is the name of the command. If this name is not the name of an existing command, an error occurs.

If an invalid property appears in the property list, an error occurs. An invalid property is one that is not supported by the named command (style properties are supported by all commands).

If the type of a property object's value does not match the type required by the property, an error occurs.

This h-code produces no result, but they are used to create l-codes. The property list given as a value is expanded to include any missing style properties and any properties specific to the named command. This process uses the property list structure created by the SET_STYLE h-code.

| | |
|---|---|
| **1** | Property List |
| **2** | String |
| | COMMAND |

### *H-code Program Execution*

After the h-code program is created (and perhaps optimized), it is executed.

If the program is optimized, a new symbol table is create, and the master l-code repository, and optimized h-code program are copied.

A pointer is set to point to the top of the copied program stack. The pointer increments until it finds an h-code, which it then executes. After the program executes its last h-code, the stack should be empty.

### *L-code Program*

L-codes are generated by the COMMAND h-code. An l-code program is a linear list of l-codes and values.

The **display**, **frame**, and **animation** commands do not add to the l-code program.

The h-code that executes the **display** command produces a structure that defines several things:

- The window width and height (or whether these should be modified at all).
- How the window reacts to size changes by the user.
- The initial zoom and pan.
- The units.

The first time the h-code is executed, the **display** command h-code is executed. The information is placed in a structure that guides the execution of the l-code. The **display** command is removed from the optimized code after this first execution.

The **frame** command h-code is used in every execution. Since it can appear anywhere in the h-code, l-code values need to be generated relative to the default frame. The l-code is executed once for each diagram or each frame of an animation. It is also executed any time the window changes size or the zoom or pan is changed. During or prior to the first execution, the rest frame values are converted to be relative to the drawing frame. Until new l-code is generated, this step can be skipped on subsequent executions.

There are three values that need transformation: coordinates, velocities, and angles (angles are treated as representing a velocity, with ±45° representing the speed of light irrespective of the frame.

The h-code that executes the **animation** command also places information in a control structure. This structure is generated only on the very first execution of the h-code—the animation command is removed from the optimized code after this first execution.

### *L-codes*

L-codes are linear sequence of objects. Each object has:

- A drawing function specific to the L-code.
- A property object with all relevant properties and values. All missing properties should be filled in with the appropriate default value.

#### Axes

Draws a set of axes. The tick mark density (if there are tick marks) depends on the current zoom. The position of the axis labels depends on the portion of the axis visible.

#### Grid

Draws the grid. The grid density depends on the current zoom.

#### HyperGrid

Draws a hyper grid. The grid density depends on the current zoom.

#### Event

Draws an event.

#### Line

Draws a line, clipped to the given clip (if supplied).

#### Worldline

Draws a wordline, clipped to the given clip (if supplied).

#### Path

Strokes and/or fills a path.

#### Label

Draws a label at a given position. The label size does not scale with zoom and, if the position falls outside the window, the labels won't be drawn at all.

### *L-code Program Execution*

The first execution sets up the window.

Each l-code object's function is executed in sequence. On the first pass, certain values need to be transformed from the default frame to the drawing frame. The value types are: coordinates, velocities, and angles. Angles are converted to velocities, transformed, and converted back, except for 45°, -45°, and equivalent, which are left as is.

From then on, only coordinates need to be transformed: from the drawing frame to pixel space using the current zoom and pan values.

## Drawing

### Zoom/Pan

TBD.

### Gamma

### Primitives

#### *Colors*

Most 2D drawing systems support RGB colors and usually alpha as well.

#### *Lines*

Usually, the 2D graphics system will provide primitives for drawing lines. We should also be able to rely on the system to perform anti-aliasing and allow us to set colors, set line thicknesses, and create dashed lines. The system should also clip to the window boundaries.

Usually, the system will not support infinite lines, so we will need to determine what portion of an infinite line fits in a window. For some lines, we support user clipping, so we may need to provide our own clipping algorithm.

#### *Tick Marks*

Tick marks are just lines, but they are infinite in number, so we will need to determine which tick marks fall inside the window. The portion of the tick mark showing my not include the point which crosses the axis, so it can get a little tricky.

Tick marks don't scale with zoom although their spacing does. Also the density of the tick marks varies with the zoom.

#### *Grids*

The density of standard and hyperbolic grids depends on the zoom and, for standard grids, the velocity of the inertial frame. They are also infinite in number, so we need to determine (as for tick marks), how many have any part showing in the window.

#### *Markers*

I'm not sure what support there is for drawing circles, squares, diamonds, and stars, which are needed for events.

These markers do not scale and disappear when the event position is clipped. We may need to do this clipping ourself.

#### *Text*

The basic system should provide all we need to create text, select the font, font weight, font style, font color, find the bounding boxes, and rotate the results. We need to compute the attachment points and offset the drawing accordingly. Text disappears when the attachment location is clipped and we may need to do this clipping ourself.

#### *Hyperbolic Curves*

Most 2D system do not support hyperbolic curves, so we will probably need to create and clip these curves ourself.

## User Interface

### Main Window

The main window will be pretty simple: A **File** menu, a **Help** menu, and a status bar.

#### *File Menu*

The File menu entries are:

- New—Create a new file on disk. The file will have the extension ".gam". If the current window already has an associated file, a new window will appear and the window will be associated with the file. If not, the current window is associated with the file. The empty file will be read and executed.

- Open—Open an existing file. If the current window already has an associated file, a new window will appear and the window will be associated with the file. If not, the current window is associated with the file. The empty file will be read and executed.
- Save Diagram—Save the current diagram as an image file. If the diagram is animated, either save the first frame or ask which frame to save.
- Save Animation—Save the animation to a video file. If the animation is unlimited, the save dialog should ask for the number of frames to output (this might be a good option anyway).
- Close—Close the current window. If this is the only window, "Close" will be grayed out.
- Exit—Exit the application.

The associated file's name will be displayed in the title bar.

The program will monitor each open file. When the file changes on disk, the file will be read and executed in the associated window(s).

#### *Help*

The Help menu entries are:

- Contents—Bring up a Help window containing (probably) an HTML help file.
- About—Display the program name, version, etc.

#### *Status Bar*

The status bar will display the rest coordinates for the cursor position. When the cursor is outside the diagram area, it will not display any coordinates.

If an animation is running, animation controls will appear in the status bar:

- Pause / Resume—When the animation is running, the button will be a pause button; when paused, it will be a resume (play) button.
- Single-step Forward—When the animation is running, this button is disabled. When the animation is paused, this button is enabled and draws the next frame. If the animation is limited and the last frame is reached, the button is disabled.
- Single-step Backward—When the animation is running, this button is disabled. When the animation is paused, this button is enabled and draws the previous frame. If the first frame is reached, the button is disabled.
- Speed Menu—A menu will allow setting the animation speed to .25X, .5X, 1X, 2X, and 4X. The desired 1X rate is 30 frames/second.

## Output

### Image Files

If the system supports writing image files, the diagram should be saveable in JPG or PNG formats.

### Animations

If the system supports writing video files, the diagram should be saveable as an animated GIF (perhaps with a warning if there are a lot of frames) and an MP4. There may be some additional options with each choice.